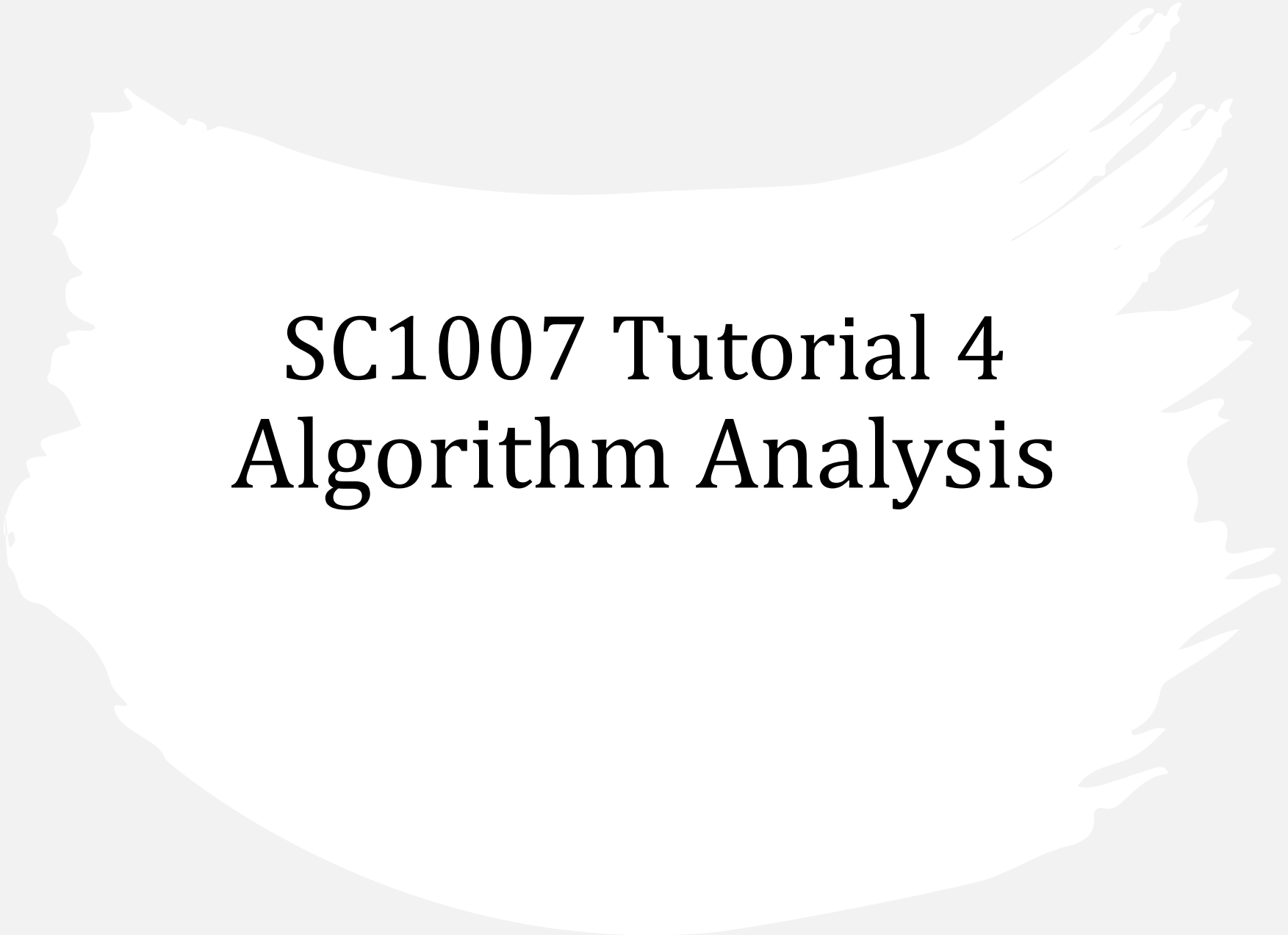




SC1007 Tutorial 4

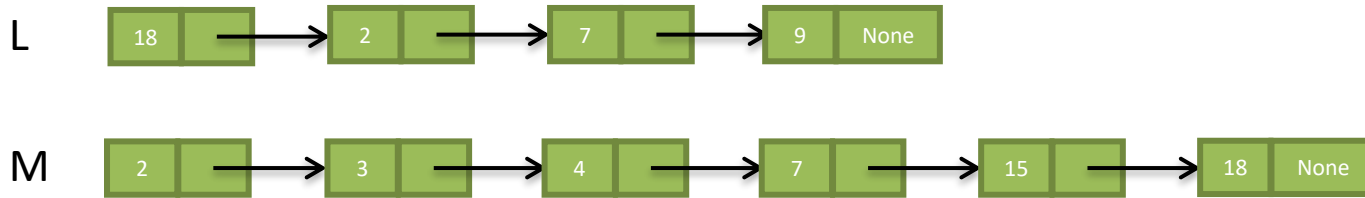
Algorithm Analysis

Question 1

The function `subset()` takes two linked lists of integers and determines whether the first is a subset of the second. Assume there are no duplicate numbers in each list.

- a) When will the worst case happen?
- b) In the worst case, how many item comparison operations will be made?
- c) Give the worst-case time complexity of `subset` as a function of the lengths of the two lists.
- d) Write down the worst-case time complexity in asymptotic notations in terms of the lengths of the two lists.

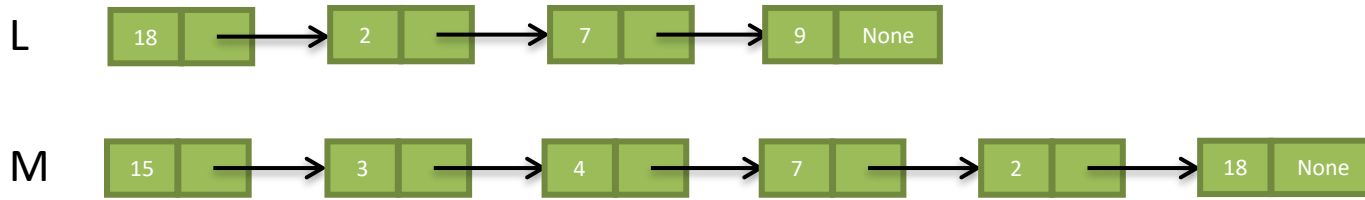
```
1  # Check whether integer X is an element of linked list Q
2  def element(X, Q):
3      found = False # Flag whether X has been found
4      while Q is not None and not found:
5          found = (Q.item == X)
6          Q = Q.next
7      return found
8
9  # Check whether L is a subset of M
10 def subset(L, M):
11     success = True # Flag whether L is a subset so far
12     while L is not None and success:
13         success = element(L.item, M)
14         L = L.next
15     return success
```



```

1  # Check whether integer X is an element of linked list Q
2  def element(X, Q):
3      found = False # Flag whether X has been found ----- C1
4      while Q is not None and not found:
5          found = (Q.item == X) } ----- C2
6          Q = Q.next
7      return found
8
9  # Check whether L is a subset of M
10 def subset(L, M):
11     success = True # Flag whether L is a subset so far
12     while L is not None and success:
13         success = element(L.item, M)
14         L = L.next
15     return success
  
```

- Node 18: $C1 + 6 \cdot C2$
- Node 2: $C1 + C2$
- Node 7: $C1 + 4 \cdot C2$
- Node 9: $C1 + 6 \cdot C2$
- Worst case1: Check for an element, e.g., 18, until the last element of M matches. The number of comparisons is 6, i.e., $|M|$
- Worst case2: The element in L is not in M, e.g., 9. The number of comparisons is 6, i.e., $|M|$
- When the size of M is large, $C1$ is negligible.



Let $|L|$ and $|M|$ indicate the length of the linked list, L and M. Assuming there are no duplicate numbers in each list, and $|L| < |M|$.

Worst case example: the first $|L|-1$ elements of L are from the last $|L|-1$ elements of M in reverse order, and the last element of L is not in M.

```

1 # Check whether integer X is an element of linked list Q
2 def element(X, Q):
3     found = False # Flag whether X has been found
4     while Q is not None and not found:
5         found = (Q.item == X) } ----- C2
6         Q = Q.next
7     return found
8
9 # Check whether L is a subset of M
10 def subset(L, M):
11     success = True # Flag whether L is a subset so far
12     while L is not None and success:
13         success = element(L.item, M)
14         L = L.next
15     return success
  
```

The number of comparisons:

The first $|L|-1$ elements in L

The last element in L

$$\begin{aligned}
 &= |M| + (|M| - 1) + \dots + (|M| - (|L| - 2)) + |M| \\
 &= |L||M| - (1 + 2 + \dots + (|L| - 2)) \\
 &= |L||M| - \frac{(1 + (|L| - 2)) \times (|L| - 2)}{2} \\
 &= |L||M| - \frac{(|L| - 1)(|L| - 2)}{2} = ?
 \end{aligned}$$

$$1 + 2 + \dots + n = \frac{(1 + n)n}{2}$$

Asymptotic Notations (review)

- Given $f(n), g(n)$
 - $\Omega(g(n))$: set of functions that grow at higher or same rate as g
 - $\Theta(g(n))$: set of functions that grow at same rate as g
 - $O(g(n))$: set of functions that grow at lower or same rate as g

$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$	$f(n) \in O(g(n))$	$f(n) \in \Omega(g(n))$	$f(n) \in \Theta(g(n))$
0	✓		
$0 < c < \infty$	✓	✓	✓
∞		✓	

Time Complexity in $|L|, |M|$

- $f(|L|, |M|) = |L||M| - \frac{(|L|-1)(|L|-2)}{2}$, $g(|L|, |M|) = |L||M|$
 - $\Omega(|L||M|)$: set of functions that grow at higher or same rate as g
 - $\Theta(|L||M|)$: set of functions that grow at same rate as g
 - $O(|L||M|)$: set of functions that grow at lower or same rate as g

$$\lim_{|M| \rightarrow \infty} \frac{f(|L|, |M|)}{g(|L|, |M|)} = \lim_{|M| \rightarrow \infty} \frac{|L||M| - \frac{(|L|-1)(|L|-2)}{2}}{|L||M|} = 1$$

The time complexity:

$$\begin{aligned} &= c_2 \times (|M| + (|M| - 1) + \dots + (|M| - |L| - 2) + |M|) = c_2 \times (|L||M| - (1 + 2 + \dots + (|L| - 2))) = \\ &c_2 \times (|L||M| - \frac{(1+(|L|-2)) \times (|L|-2)}{2}) = c_2 \times (|L||M| - \frac{(|L|-1)(|L|-2)}{2}) = \Theta(|L||M|) \end{aligned}$$

Question 2

- Find the number of print used in the following functions. Write down its time complexity in Θ notation in terms of N .

```
def Q2a(N):  
    j = 1  
    while j <= N:  
        k = 1  
        while k <= N:  
            print("SC 1007")  
            k *= 2  
        j *= 3
```

```
def Q2b(N):  
    if N > 0:  
        for i in range(N):  
            print("SC 1007")  
        Q2b(N - 1)  
        Q2b(N - 1)
```

```
def Q2a(N):
    j = 1
    while j <= N:
        k = 1
        while k <= N:
            print("SC 1007")
            k *= 2
        j *= 3
```

N	Number of print	k value when inner loop stops	j value when outer loop stops
1		2^1	3^1
10		2^4	3^3
100		2^7	3^5

- For the inner loop:

$$2^{K-1} \leq N \leq 2^K$$

$$(K - 1) \leq \log_2 N \leq K$$

$$K \leq \log_2 N + 1 \leq K + 1$$

$$K = \lfloor \log_2 N \rfloor + 1$$

- For the outer loop:

$$3^{J-1} \leq N \leq 3^J$$

$$(J - 1) \leq \log_3 N \leq J$$

$$J \leq \log_3 N + 1 \leq J + 1$$

$$J = \lfloor \log_3 N \rfloor + 1$$

- The number of print is $JK = (\lfloor \log_3 N \rfloor + 1)(\lfloor \log_2 N \rfloor + 1)$

Common Complexity Classes

Order of Growth	Class	Example
1	Constant	Finding midpoint of an array
$\log_2 n$	Logarithmic	Binary Search
n	Linear	Linear Search
$n \log_2 n$	Linearithmic	Merge Sort
n^2	Quadratic	Bubble Sort
n^3	Cubic	Matrix Inversion (Gauss-Jordan Elimination)
2^n	Exponential	Fibonacci Sequence (recursive)
$n!$	Factorial	Travelling Salesman Problem

```
def Q2b(N):
    if N > 0:
        for i in range(N):
            print("SC 1007")
        Q2b(N - 1)
        Q2b(N - 1)
```

N	Number of printf	inner loop stops		outer loop stops	
1	1*1	2:	2^1	3:	3^1
10	3*4	16:	2^4	27:	3^3
100	5*7	128:	2^7	243:	3^5

- Time number of print is $JK = (\lfloor \log_3 N \rfloor + 1)(\lfloor \log_2 N \rfloor + 1)$
- $\lim_{N \rightarrow \infty} \frac{(\lfloor \log_3 N \rfloor + 1)(\lfloor \log_2 N \rfloor + 1)}{(\log_2 N)^2} = \frac{1}{\log_2 3}$
- The time complexity is $\Theta((\log_2 N)^2)$

```
def Q2b(N):
    if N > 0:
        for i in range(N):
            print("SC 1007")
        Q2b(N - 1)
        Q2b(N - 1)
```

- $W_1 = 1$
- $W_2 = 2 + W_1 + W_1$
- $W_N = N + W_{N-1} + W_{N-1}$
 $= N + 2W_{N-1}$
 $= N + 2(N - 1 + 2W_{N-2})$
 $= N + 2(N - 1) + 2^2W_{N-2}$
 $= N + 2(N - 1) + 2^2(N - 2) + \dots + 2^{N-1}(W_1) = \sum_{t=0}^{N-1} 2^t(N - t)$
 $= N \sum_{t=0}^{N-1} 2^t - \sum_{t=0}^{N-1} 2^t t$
 $= N \sum_{t=0}^{N-1} 2^t - 2 \sum_{t=1}^{N-1} 2^{t-1} t = ?$

Series

- Geometric Series

$$G_n = \frac{a(1 - r^n)}{1 - r} \quad \sum_{t=0}^{N-1} 2^t = \frac{1 - 2^N}{1 - 2} = 2^N - 1$$

- Arithmetic Series

$$A_n = \frac{n}{2}[2a + (n - 1)d] = \frac{n}{2}[a_0 + a_{n-1}]$$

- Arithmetico-geometric Series

$$\sum_{t=1}^k t 2^{t-1} = 2^k(k - 1) + 1 \quad \sum_{t=1}^{N-1} 2^{t-1} t = 2^{N-1}(N - 2) + 1$$

- Faulhaber's Formula for the sum of the p-th powers of the first n positive integers

$$\sum_{k=1}^n k^2 = \frac{n(n + 1)(2n + 1)}{6}$$

$$\sum_{k=1}^n k^3 = \frac{n^2(n + 1)^2}{4}$$

*Derivation is in note section 0.7.4.1

The number of print: $W_N = N \sum_{t=0}^{N-1} 2^t - 2 \sum_{t=1}^{N-1} 2^{t-1} t =$

Common Complexity Classes

Order of Growth	Class	Example
1	Constant	Finding midpoint of an array
$\log_2 n$	Logarithmic	Binary Search
n	Linear	Linear Search
$n \log_2 n$	Linearithmic	Merge Sort
n^2	Quadratic	Bubble Sort
n^3	Cubic	Matrix Inversion (Gauss-Jordan Elimination)
2^n	Exponential	Fibonacci Sequence (recursive)
$n!$	Factorial	Travelling Salesman Problem

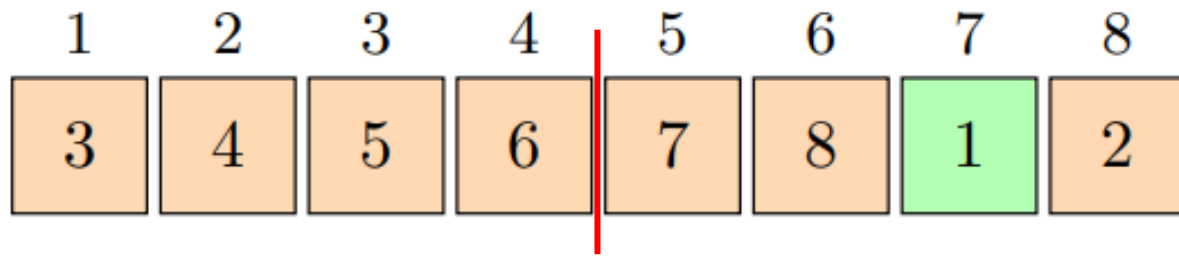
```
def Q2b(N):
    if N > 0:
        for i in range(N):
            print("SC 1007")
        Q2b(N - 1)
        Q2b(N - 1)
```

- $W_1 = 1$
- $W_N = N + W_{N-1} + W_{N-1}$
 $= N + 2W_{N-1}$
 $= N + 2(N - 1 + 2W_{N-2})$
 $= N + 2(N - 1) + 2^2W_{N-2}$
 $= N + 2(N - 1) + 2^2(N - 2) + \dots + 2^{N-1}(W_1) = \sum_{t=0}^{N-1} 2^t(N - t)$
 $= N \sum_{t=0}^{N-1} 2^t - 2 \sum_{t=0}^{N-1} 2^{t-1}t = 2^{N+1} - 2 - N$
- $\lim_{N \rightarrow \infty} \frac{2^{N+1} - 2 - N}{2^N} = 2$
- The time complexity is $\Theta(2^N)$

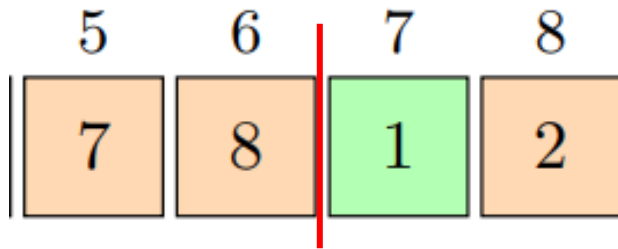
Question 3

- A sequence, $x_1, x_2, \dots, x_n$, is said to be cyclically sorted if the smallest number in the sequence is x_i for some i , and the sequence, $x_i, x_{i+1}, \dots, x_n, x_1, x_2, \dots, x_{i-1}$ is sorted in increasing order. Design an algorithm to find the minimal element in the sequence in $O(\log n)$ time. What is the worst-case scenario?

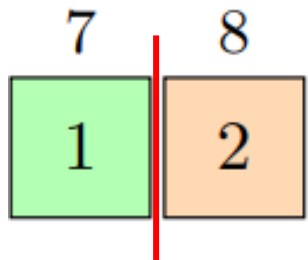
1	2	3	4	5	6	7	8
6	7	8	1	2	3	4	5
1	2	3	4	5	6	7	8
3	4	5	6	7	8	1	2



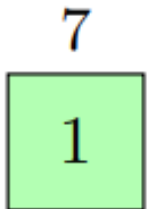
middle = 6, middle > last, i.e., 2.
The minimum is in the second half



middle = 8, middle > last, i.e., 2
The minimum is in the second half



middle = 1, middle < last, i.e., 2
The minimum is in the first half



Only one element

- The number of comparisons (line 3)

- $W_1 = 1$
- $W_2 = 2$
- $W_4 = 3$
- $W_8 = 4$
-

- Time complexity

$$T(n) = T\left(\frac{n}{2}\right) + c$$

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + c$$

$$T(n) = T\left(\frac{n}{4}\right) + 2c$$

$$\vdots$$

$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$

```

1  def find_minimum(array, m, n):  3 usages
2      if m == n:
3          return array[m]
4      else:
5          middle = (m + n) // 2
6          if array[middle] < array[n]:  # in the first half
7              return find_minimum(array, m, middle)
8          else:  # in the second half
9              return find_minimum(array, middle + 1, n)
10
11 array = [3, 4, 5, 6, 7, 8, 1, 2]
12 minimum = find_minimum(array, m: 0, len(array) - 1)
13 print(f"the minimum value is {minimum}")

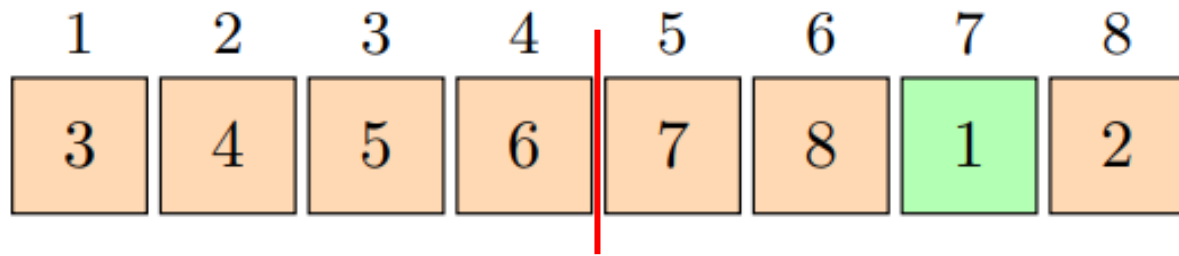
```

$$0 < n/2^k \leq 1$$

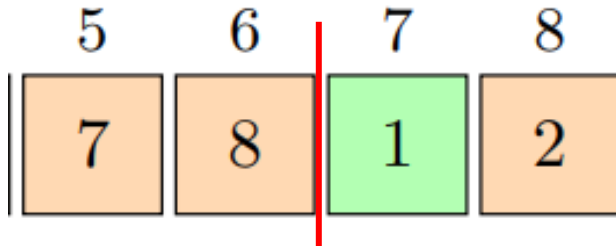
$$k \geq \log_2 n$$

$$k = \lceil \log_2 n \rceil$$

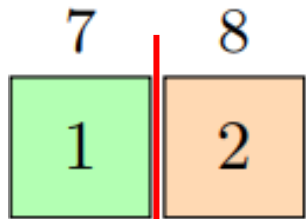
$$T(n) = T\left(\frac{n}{2^k}\right) + kc = (\lceil \log_2 n \rceil + 1)c = \Theta(\log_2 n)$$



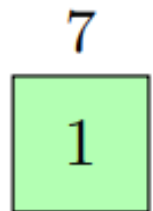
middle = 6, middle > last, i.e., 2.
The minimum is in the second half



middle = 8, middle > last, i.e., 2
The minimum is in the second half



middle = 1, middle < last, i.e., 2
The minimum is in the first half



Only one element

Given n,
worst case?

Given n, all cases have to run the same number of comparisons until only one element is left.